

Machine Learning



Dr. Ahmed Hesham Mostafa
Faculty of Computers and AI
Helwan University
Lecture 4

Course Staff: Instructors (Short Bio)

- **Dr. Ahmed Hesham Mostafa**
- Bachelor in Computer Science *from* Helwan University
- Masters in Computer Science (Malware Analysis & Artificial Intelligence) *from* Helwan University
- PhD. in Computer Science (Computer Vision & Artificial Intelligence) *from* Helwan University
- E-mail : ahmedheshamostafa@gmail.com

Agenda

- Why Can't We Use Linear Regression To Solve A Classification Problem?
- Logistic Regression
- SoftMax Classifier
- Entropy

Why Can't We Use Linear Regression To Solve A Classification Problem?

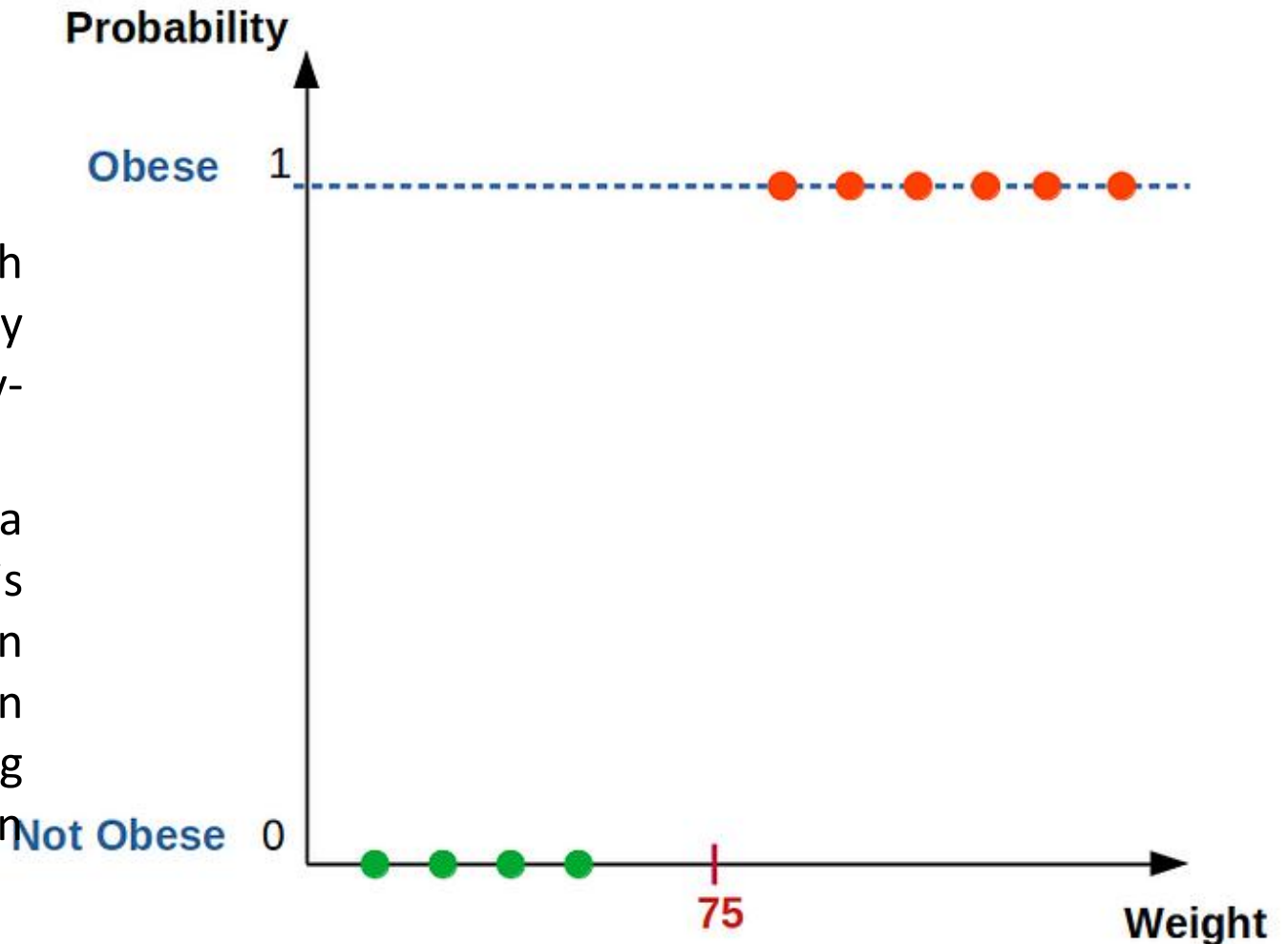
- Linear regression and Logistic regression are two of the earliest machine learning algorithms that came into existence. Both of them are supervised machine learning algorithms.
- Linear regression is used when the output data is continuous in nature. While Logistic regression is used when the output data is discrete in nature.
- But if we can solve a classification problem using Linear regression, then why do we need Logistic regression in the very first place?

Why Can't We Use Linear Regression To Solve A Classification Problem?

- You have a dataset that consists of the weights of people.
- To gain speedy and accurate results, you take the help of machine learning algorithms to identify if a person is obese or not based on his weight.
- From the overview, you can easily identify that this is a binary classification problem where if a person is above a certain weight, let's say 75 kg, then he is considered **obese (1)** and if a person is below 75kg, then he is considered **not obese (0)**

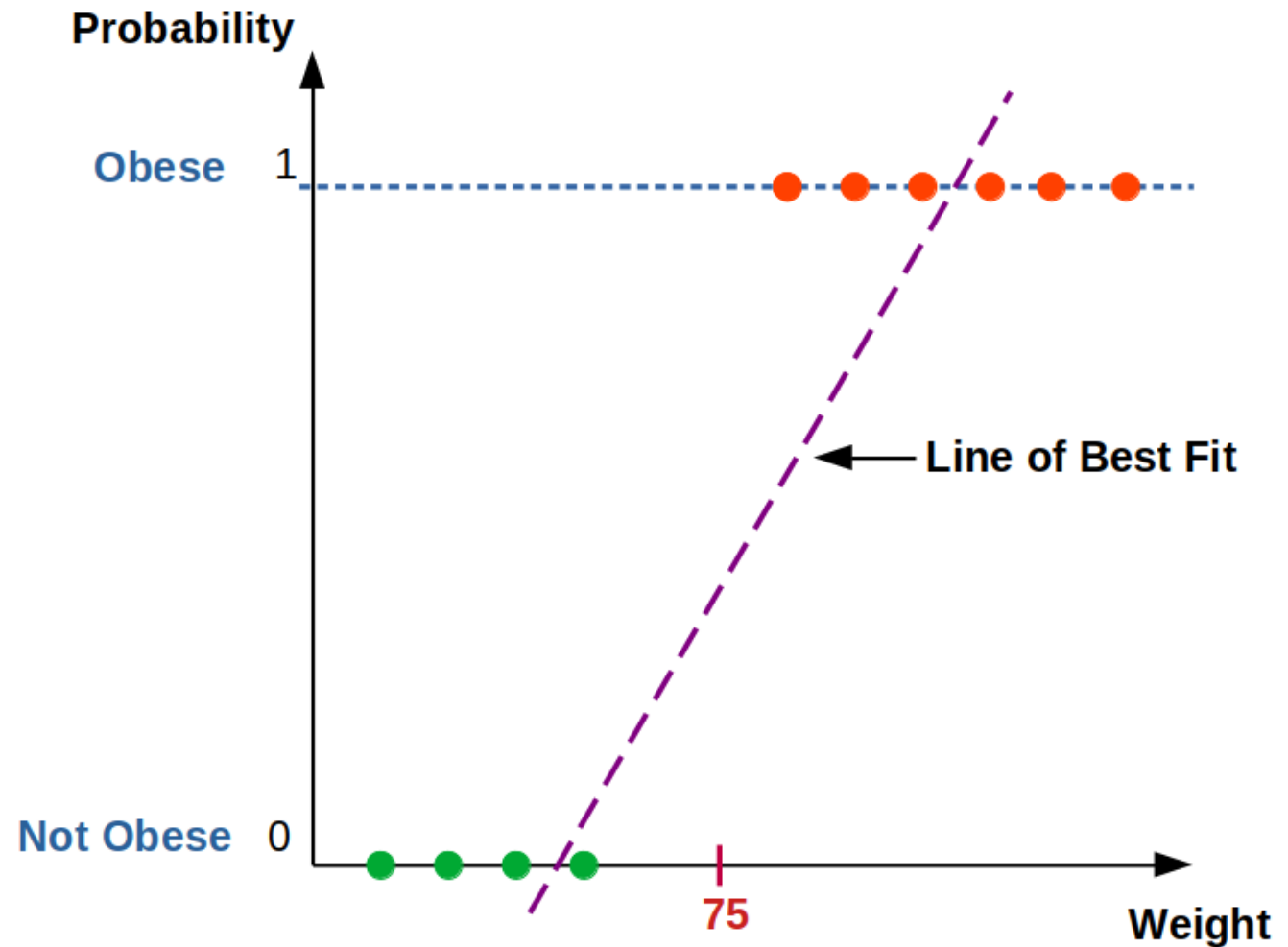
Will Linear Regression Help You To Classify If A Person Is Obese Or Not?

- You plot a 2-dimensional graph with weight on the x-axis and the probability that a person is obese or not on the y-axis.
- You classify the sample data in such a way that if a person is below 75kg, he is considered as not obese and if a person is above 75kg, he is considered obese. In short, we are categorizing or classifying people as obese or not obese based on their weight.



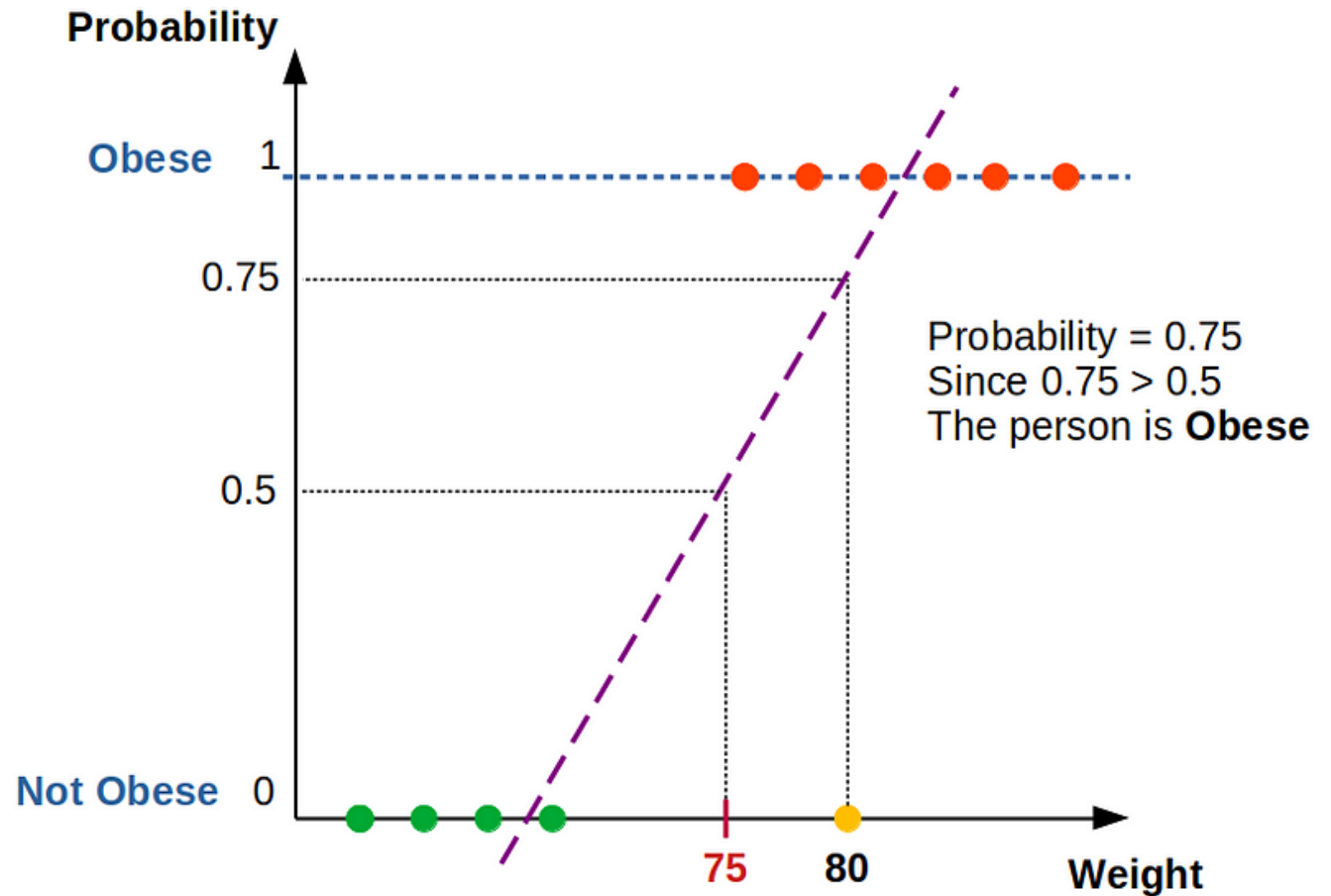
Will Linear Regression Help You To Classify If A Person Is Obese Or Not?

- So after classifying a sample dataset, we draw a line of best fit through the data points. The line of best fit is drawn in such a way that the distance between the line and all the data points is minimum.



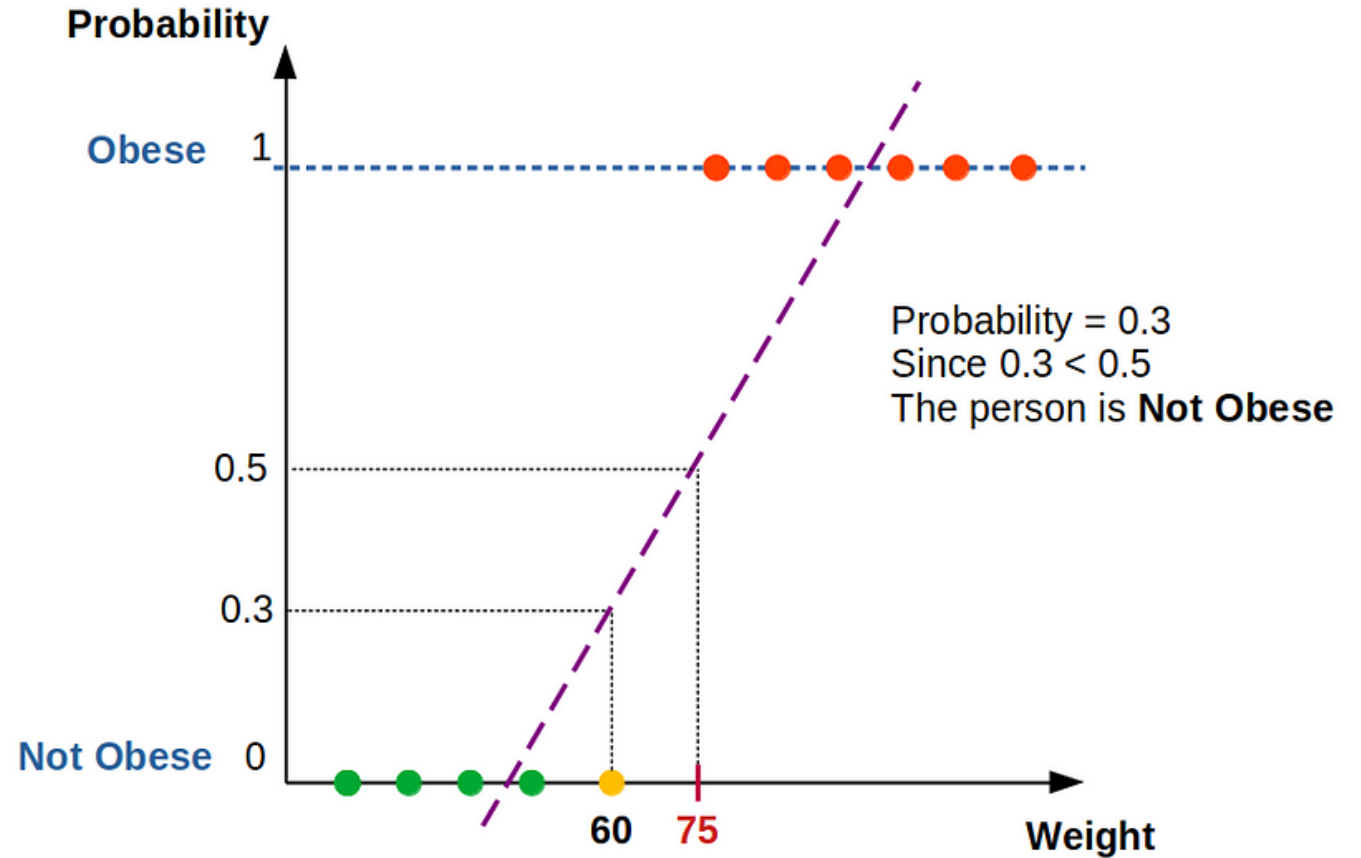
Will Linear Regression Help You To Classify If A Person Is Obese Or Not?

- Therefore, for a person who weighs 80 kg, the probability will be greater than 0.5 and hence he will be classified as obese.



Will Linear Regression Help You To Classify If A Person Is Obese Or Not?

- And for a person who weighs 60 Kg, the probability will be less than 0.5 and hence he will be classified as not obese.

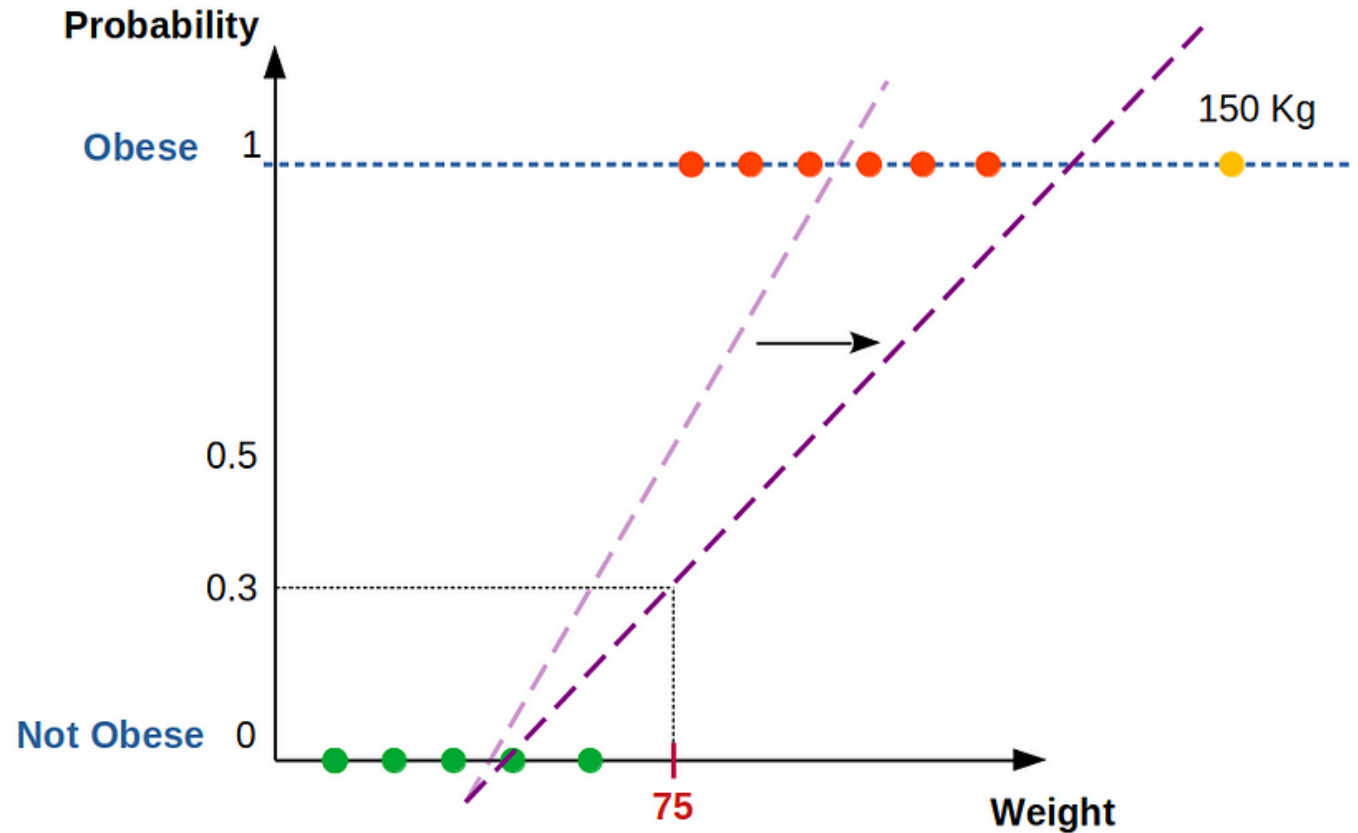


Will Linear Regression Help You To Classify If A Person Is Obese Or Not?

- In our example, we can see that Linear regression is able to classify a person as obese or not obese. Then why do we need logistic regression? Can't we use Linear regression to perform further classifications?
- **Always remember that Linear Regression is highly affected by the inclusion of an outlier.**

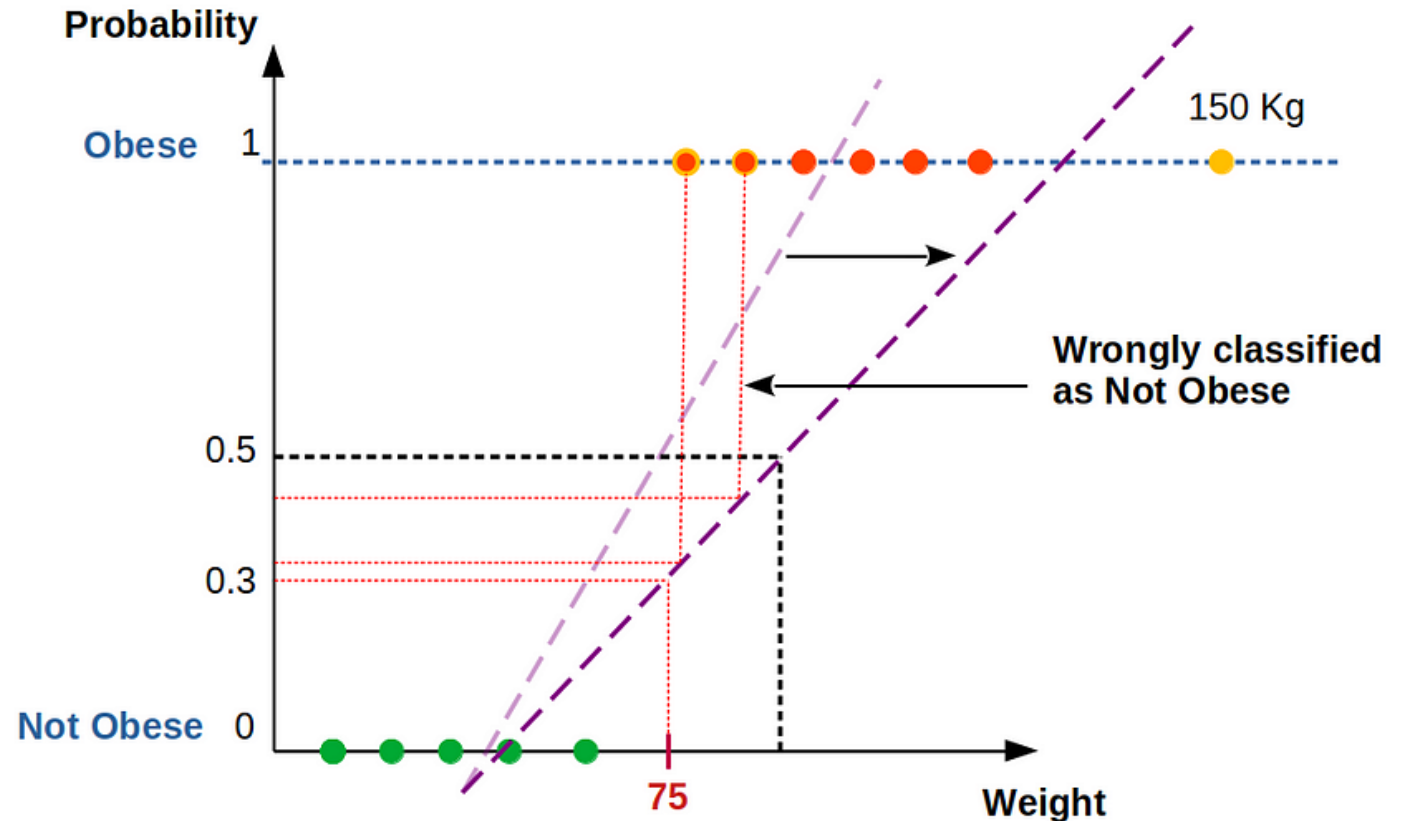
Classification Problems with liner regression

- Suppose, during our initial classification, an outlier gets added to the graph. Say 150 Kg.
- Because of this outlier, the position of the line of best fit completely changes.



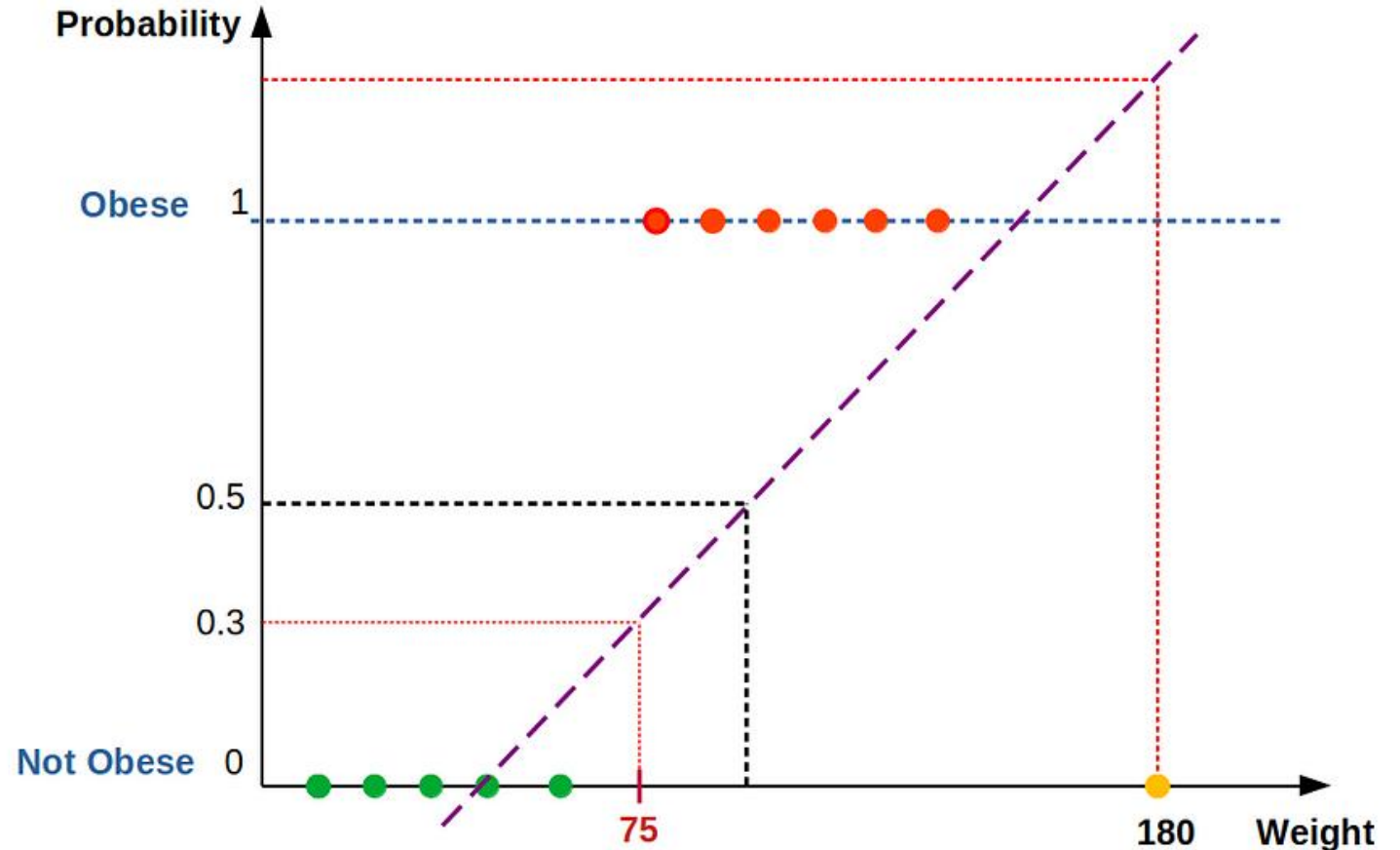
Classification Problems with liner regression

- As our problem statement, if a person is above 75 Kg, we are considering that person as obese. But now, because of the new line of best fit, the probability associated with 75 Kg is less than 0.5 Hence it will be classified as not obese which is absolutely wrong.
- And with that, all the data points whose probability comes below 0.5 as per the new line of best fit will be considered as not obese.



Classification Problems with linear regression

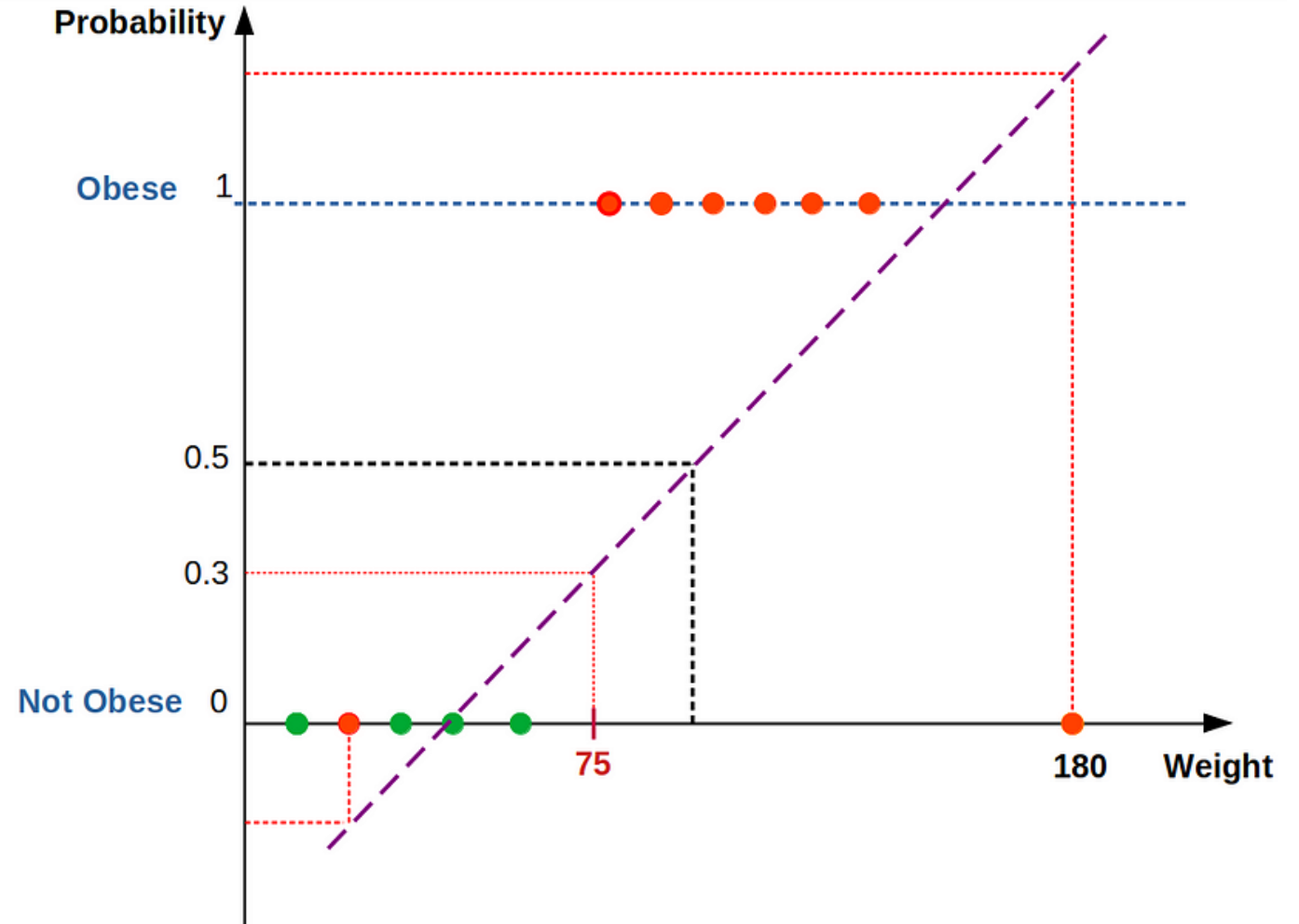
- Therefore, just by the inclusion of one outlier, the entire result goes for a toss and the error rate increases drastically.
- There is another problem with Linear regression.
- Suppose, a new weight gets added to our data. Let the new weight be 180 kg.



In this case, if we try to find out the probability of a person being obese or not with respect to this weight, then the probability comes out to be greater than 1. This is an absolutely absurd result since the probability of any event cannot be greater than 1.

Classification Problems with linear regression

- Similarly, there can be a scenario where the output probability associated with weight will be less than zero. Which again does not make any sense.



Why Can't We Use Linear Regression To Solve A Classification Problem?

- Linear regression is a great algorithm but it is highly impacted by outliers. Hence we cannot use it to solve a classification problem.
- We need an algorithm that absorbs the effects of outliers without impacting the final output.
- So we need another algorithm to absorb the effects of outliers without impacting the final output and that introduces the Logistic regression

Logistic Regression

- **Logistic regression** is a **supervised machine learning algorithm** used for **classification tasks** where the goal is to predict the probability that an instance belongs to a given class or not.

Logistic Regression

- As we know from linear regression that multi-linear function to the input variables X . is $z = (\sum_{i=1}^n w_i x_i) + b$
- Here x_i is the i th observation of X , $w_i = [w_1, w_2, w_3, \dots, w_m]$ is the weights or Coefficient, and b is the bias term also known as intercept. simply this can be represented as the dot product of weight and bias.
- $z = w \cdot X + b$

Logistic Regression

- But how to make the output of this function $z = w \cdot X + b$ to be between 0 to 1 and as we know the x is natural number and output of z will be always between $-\infty$ to $+\infty$
- From other word we need to make the output of the function between 0 to 1 instead of $-\infty$ to $+\infty$
- For this reason we will introduce the probability, odds and the log odds (logit)

Odds V.S Probability

- ***Odds*** are the *ratio of something happening to something not happening.*
- ***Probability*** is the *ratio of something happening to everything that could happen.*

Odds V.S Probability

- The odds equals the probability that $Y=1$ divided by the probability that $Y=0$.
- For example, if the probability that $Y = 1$ is 0.8 (or that there's an 80% probability of $Y=1$), then the probability that $Y=0$ is 1-0 or 0.2 (remember, Y can only be 0 or 1, so the probability that $Y=0$ is 1-[probability that $Y=1$]).
- Using these numbers, we can calculate the odds as the ratio of these two numbers:
- Odds = $P(Y=1)/P(Y=0) = 0.8/0.2 = 4$

Odds V.S Probability

- In this case, the odds is 4. You will often hear people refer to this as 4:1 odds, which you would read as "four to one odds." Now that we know how odds are related to probability

- $\frac{p(y=1)}{1-p(y=1)} = Odds$ or $\frac{p}{1-p} = Odds$

- Or

- $\frac{p(y=1)}{p(y=0)} = Odds$

Odds ratio and Logit

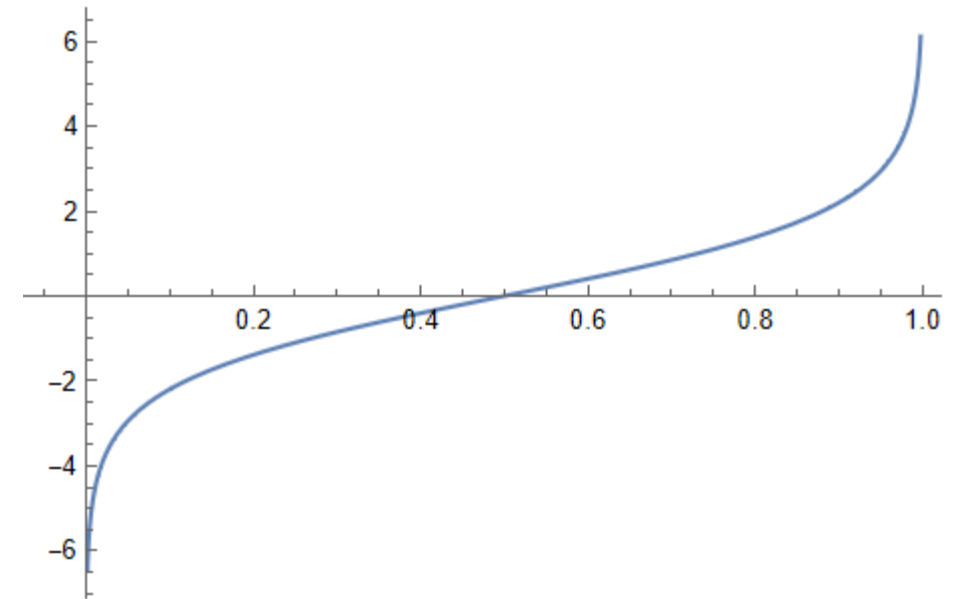
- One problem with working with odds directly is that reciprocal odds yield non-intuitive values (a reciprocal is just a fraction “flipped” around, i.e. “ y/x ” is the reciprocal of “ x/y ”.) For example, if the probability of success in a given experiment is 40%, then the odds of success are $4/6$, or $\sim .667$. The odds of losing, on the other hand, are the reciprocal of the odds of success; $6/4$, or 1.5.
- Intuitively, we can see that these odds are in some way “symmetrical” when we view them as fractions, but when rendered into decimal form this is less clear. It’s not obvious how to “center” 1.5 and .667 on a scale that would make it easy for an observer to understand that these values are related reciprocally.

Odds ratio and Logit

- $\text{Log}(4/6) = \log(4) - \log(6) = -0.405$
- Log And what is the logarithm of the inverse odds, 6/4? Well, by applying the same quotient rule we get
- $\text{Log}(6/4) = \log(6) - \log(4) = +0.405$
- This gives us a very nice scale that is “centered” at zero on the number line.

Logit Function

- Now let's take a look at what the log-odds look like for any conceivable set of odds. We call this formulation the “logit” function (an abbreviation of “logistic unit.”)
- $\log\left(\frac{p}{1-p}\right) = \log(Odds) = \textit{logit}$
- The logit function maps the probability of an event (expressed here in decimal form on the x-axis) to its log-odds on the y-axis.

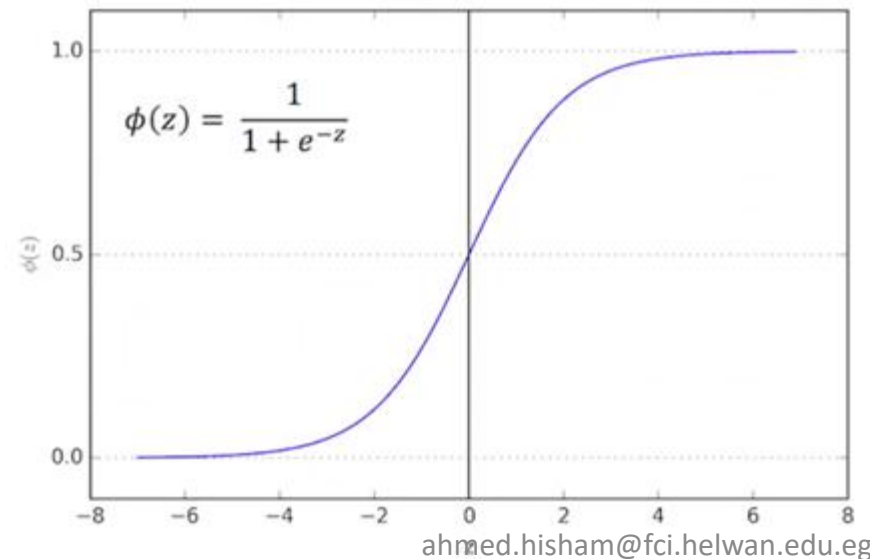


Logit function

- Note that the range output of **propapbility** is **0 to 1**
- Not that the output range of **odd** is **0 to $+\infty$** because if dominator is 0 ($1/0$) will equal
- Note that the output range for **logit** will be **$-\infty$ to $+\infty$**

sigmoid function

- The inverse of the logit function is the **sigmoid** function. That is, if you have a probability p , $\text{sigmoid}(\text{logit}(p)) = p$. The sigmoid function maps arbitrary real values back to the range $[0, 1]$. We can also say sigmoid function as the generalized form of logit function.



Logistic Regression

- So this remember us what is our original problem
- we need to make the output of the function $z = w \cdot X + b$ between 0 to 1 instead of $-\infty$ to $+\infty$
- As we see that the output range for logit is $-\infty$ to $+\infty$
- As we can say that
- $z = \text{logit}$
- $z = \log\left(\frac{p}{1-p}\right)$

Logistic Regression

- $\log \left[\frac{p(x)}{1-p(x)} \right] = z$
- $\log \left[\frac{p(x)}{1-p(x)} \right] = w \cdot X + b$
- $\frac{p(x)}{1-p(x)} = e^{w \cdot X + b}$... Exponentiate both sides
- $p(x) = e^{w \cdot X + b} \cdot (1 - p(x))$
- $p(x) = e^{w \cdot X + b} - e^{w \cdot X + b} p(x)$

Logistic Regression

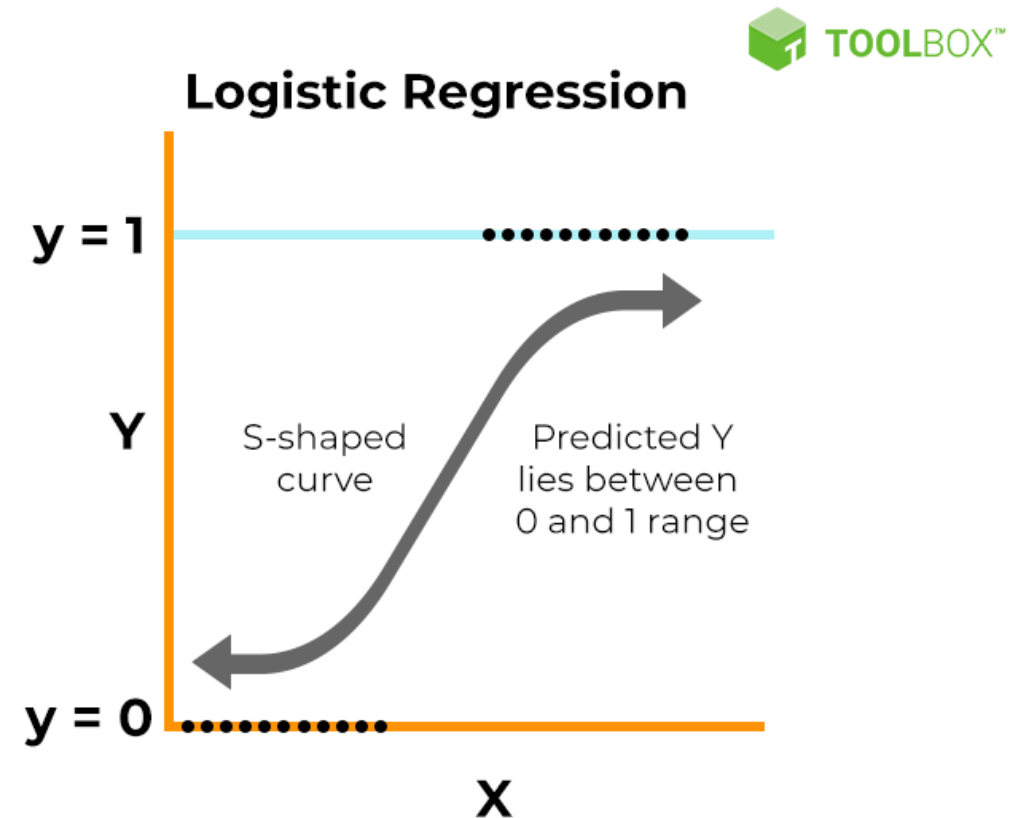
- $p(x) + e^{w \cdot X + b} p(x) = e^{w \cdot X + b}$
- $p(x)(1 + e^{w \cdot X + b}) = e^{w \cdot X + b}$
- $p(x) = \frac{e^{w \cdot X + b}}{1 + e^{w \cdot X + b}}$
- as $z = w \cdot X + b$
- $p(x) = \frac{e^z}{1 + e^z}$

Logistic Regression

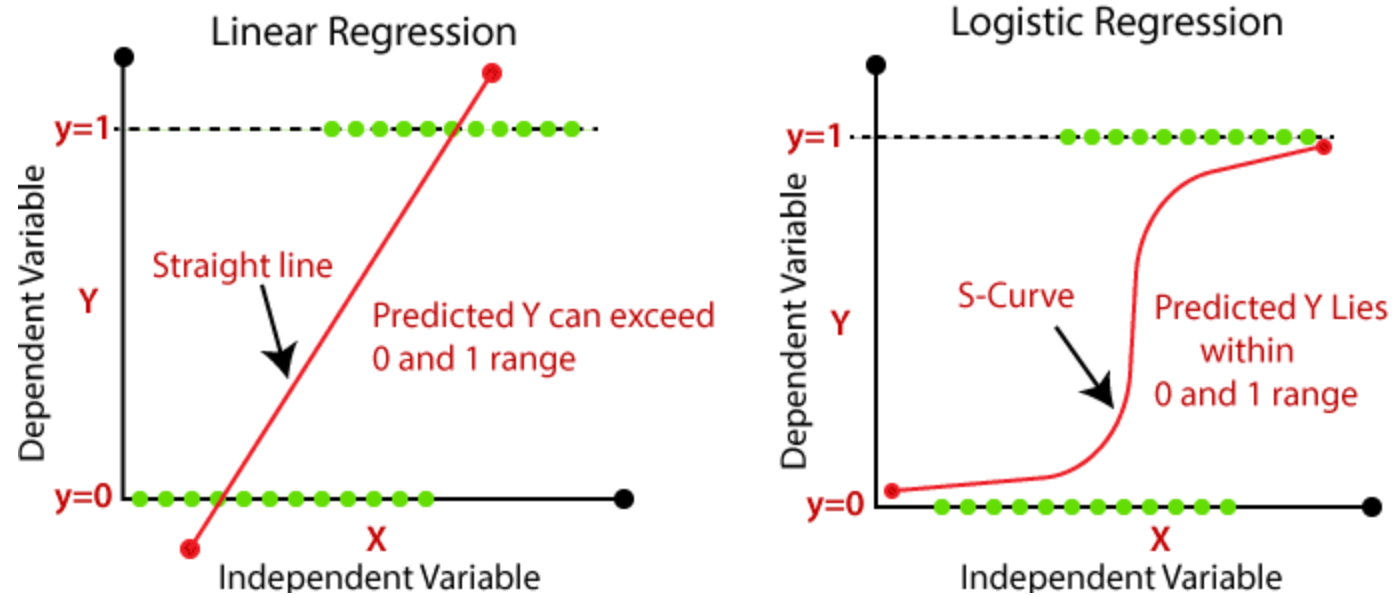
- $p(x) = \frac{e^z}{1+e^z}$
- *if we divide numerator and dominator bye^z*
- $p(x) = \frac{1}{\frac{1}{e^z}+1} = \frac{1}{e^{-z}+1}$ *// remember Sigmoid function*
- $p(y = 1|X) = \frac{e^{w \cdot X + b}}{1+e^{w \cdot X + b}} = \frac{1}{1+e^{-w \cdot X + b}}$ *wher p is the predicted probability that the example belongs to class 1.*

What is Logistic Regression?

- Logistic regression is used for binary classification where we use **sigmoid function**, that takes input as independent variables and produces a probability value between 0 and 1.
- For example, we have two classes Class 0 and Class 1 if the value of the logistic function for an input is greater than 0.5 (**threshold value**) then it belongs to Class 1 otherwise it belongs to Class 0. It's referred to as regression because it is the extension of linear regression but is mainly used for classification problems.



Logistic Regression V.S Linear Regression



Types of Logistic Regression

- 1.Binomial:** In binomial Logistic regression, there can be only two possible types of the dependent variables, such as 0 or 1, Pass or Fail, etc.
- 2.Multinomial:** In multinomial Logistic regression, there can be 3 or more possible unordered types of the dependent variable, such as “cat”, “dogs”, or “sheep”

Cost Function of a Logistic Regression

- The Cost Function of a Logistic Regression cannot use MSE because our prediction function is non-linear (due to sigmoid transform).
- Therefore we use a cost function called Cross-Entropy, also known as Log Loss.

Cost Function of a Logistic Regression

- Logistic Regression cannot use MSE because the output of logistic regression is small values between $[0,1]$ and when squaring these values the results are will be very small
- Ex: $(0.05)^2=0.0025$
- While when we take the log of output the result will be very large
- EX: $\log(1/0.05)= 2.342$

cross-entropy loss

- The cross-entropy loss is derived from the concept of entropy in information theory.
- Entropy measures the uncertainty or randomness of a random variable.
- In the context of classification, the cross-entropy loss quantifies the performance of a classification model by comparing the predicted probability distribution with the true distribution of labels.

Entropy

- Entropy is a measure of uncertainty or disorder in a set of data. In information theory, entropy is defined as: $H(X) = -\sum_{i=1}^n P(x_i) \log_2 P(x_i)$
- Where: - $H(X)$ is the entropy of the random variable X . - $P(x_i)$ is the probability of the i th outcome. - n is the number of possible outcomes.
- Entropy measure the surprise for occurring for an event for example if the probability of weather to rain is very low say $p(x) = 0.05$ so if the weather is rain there is a very big surprise so the to represent the surprise we want to enlarge the $p(x) = 0.05$ to a large number so take the log for $1/p(x)$
- So entropy for weather is rain = $\log(1/p(1)) = \log(1/0.05) = 4.322$
- Entropy for weather is not rain is = $\log(1/.95) = 0.074$
- So if rained I will be very surprised while is not rain I will not be surprised at all

Entropy

	Rain	Not Rain
Probability $P(X = x)$	$P(x_1) = 0.05$	$P(x_2) = 0.95$
Surprise $\log_2\left(\frac{1}{P(x)}\right)$	4.322	0.074

- So total surprise we can get from the weather is rain or not =
- Total surprise for three days where it rain for first day and not rain for the two other day =
surprise(rain)+ surprise(not rain)+ surprise(not rain)
- = surprise(rain)+ 2* surprise(not rain)
- But if we want to estimate the total surprise, we can get for weather for 50 days and we don't know how the whether will be in these 50 days so we estimate the number of days can be rain and not rain based on their probability so
- Total surprise = $50 * 0.05 * \text{surprise}(\text{rain}) + 50 * .95 * \text{surprise}(\text{not rain})$
- = $50 * .05 * 4.322 + 50 * .95 * 0.074 = 14.32$
- If we divide the two side on 50 we can get the average
- Avg(total surprise) = $50 * .05 * 4.322 + 50 * .95 * 0.074 = 14.32 = 0.2864$
- So the average surprise is 0.2864

Entropy

- Avg (total surprise) = $\frac{\cancel{50} * P(x_1) * \log_2(\frac{1}{P(x_1)}) + \cancel{50} * P(x_2) * \log_2(\frac{1}{P(x_2)})}{\cancel{50}}$
- So entropy is the average of surprise
- Entropy (X) = $\sum_{i=1}^N p(x_i) * \log_2(\frac{1}{P(x_i)})$
- = $\sum_{i=1}^N p(x_i) * (\log_2 1 - \log_2 P(x_i))$ /properties of log
- = $\sum_{i=1}^N -p(x_i) * \log_2 P(x_i)$ as $\log 1 = 0$
- = $-\sum_{i=1}^N p(x_i) \log_2 P(x_i)$

Entropy and Loss

- But what is the relation between entropy and the loss
- When we classify element and we expect the output for input data sample will be belong to class 1 and so the output probability will be high but we surprised that the output of probability is very low for example 0.001 that mean it belong to class 0
- So the surprise will be high and mean the loss is high

Cross-Entropy Loss

- Cross-Entropy Loss Formula for Binary Classification:**
- In binary classification, where there are only two possible classes (e.g., 0 and 1), the cross-entropy loss function is commonly used. Given the true label y and the predicted probability $\hat{y}$, the cross-entropy loss is calculated as:
- $$L(y, \hat{y}) = -(y \log(\hat{y}) + (1 - y) \log(1 - \hat{y}))$$
- Where: - y is the true label (0 or 1). - $\hat{y}$ is the predicted probability of the positive class.

Categorical Cross-Entropy

- Cross-Entropy Loss Formula for Categorical Classification:**
- In categorical classification, where there are multiple classes, the cross-entropy loss function is used. Given the true label y (represented as a one-hot encoded vector) and the predicted probability distribution $\hat{y}$, the cross-entropy loss is calculated as:
 - $L(y, \hat{y}) = - \sum_{i=1}^n y_i \log(\hat{y}_i)$
- Where: - y is the true label (one-hot encoded vector). - $\hat{y}$ is the predicted probability distribution. - n is the number of classes. - y_i is the i th element of the true label vector y . - $\hat{y}_i$ is the i th element of the predicted probability vector $\hat{y}$.

Cross-entropy loss

- Cross-entropy loss values range from 0 to infinity, with lower values indicating better model performance. A cross-entropy loss of 0 indicates perfect prediction accuracy, where the predicted probabilities match the true labels exactly.
- However, it's important to note that cross-entropy loss values should be interpreted in the context of the specific problem and dataset. A lower cross-entropy loss does not necessarily guarantee better performance on unseen data, as it may indicate overfitting. Techniques like regularization and early stopping should be employed to prevent overfitting and ensure the model's generalization capabilities.

How Logistic Regression Work

- Now return to our concern and show how logistic regression work
- ! Here are the steps to implement a logistic regression classifier with cross-entropy loss:
- 1. ****Initialize Parameters****: - Initialize the weights (w) and bias (b) parameters of the logistic regression model. These parameters can be initialized randomly or to zero.
- 2. ****Compute Linear Transformation****: - For each training example i , compute the linear transformation $z^{(i)} = w^T x^{(i)} + b$, where $x^{(i)}$ is the feature vector of the i -th example.
- 3. ****Apply Sigmoid Function****: - Apply the sigmoid function $\sigma(z^{(i)}) = \frac{1}{1+e^{-z^{(i)}}}$ to each $z^{(i)}$ to obtain the predicted probability $\hat{y}^{(i)}$ that the example belongs to class 1.
- 4. ****Compute Loss Function****: - Calculate the cross-entropy loss function over the entire training set:

How Logistic Regression Work

- $J(w, b) = -\frac{1}{N} \sum_{i=1}^N [y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})]$
- where N is the number of training examples, $y^{(i)}$ is the true label (0 or 1) for the i -th example, and $\hat{y}^{(i)}$ is the predicted probability of class 1 for the i -th example.
- 5. ****Compute Gradients****: - Compute the gradients of the loss function with respect to the parameters w and b :
 - $\frac{\partial J}{\partial w} = \frac{1}{N} \sum_{i=1}^N (\hat{y}^{(i)} - y^{(i)}) x^{(i)}$
 - $\frac{\partial J}{\partial b} = \frac{1}{N} \sum_{i=1}^N (\hat{y}^{(i)} - y^{(i)})$

Note in Gradient Lecture we will show how to compute the gradient for MSE, Cross entropy , categorical cross entropy and the Softmax

How Logistic Regression Work

- 6. ****Update Parameters****: - Update the parameters w and b using gradient descent or any other optimization algorithm:
 - $w := w - \alpha \frac{\partial J}{\partial w}$
 - $b := b - \alpha \frac{\partial J}{\partial b}$
 - where α is the learning rate.
- 7. ****Repeat****: - Iterate steps 2-6 for a fixed number of iterations or until convergence criteria are met.

How Logistic Regression Work

- 8. ****Prediction****: - Use the trained parameters w and b to make predictions on new examples:
- $\hat{y}^{(new)} = \sigma(w^T x^{(new)} + b)$
- where $x^{(new)}$ is the feature vector of the new example.

Multinomial Logistic Regression: Softmax

- Target variable can have 3 or more possible types which are not ordered (i.e. types have no quantitative significance) like “disease A” vs “disease B” vs “disease C”.
- In this case, the softmax function is used in place of the sigmoid function.

Multinomial logistic regression

- Multinomial logistic regression, also known as softmax regression, is an extension of binary logistic regression to handle multiple classes.
- It's a classification algorithm used when the dependent variable is categorical with more than two levels.

How Softmax regression works

- ****Model Representation****: - Given n features $x_1, x_2, \dots, x_n$, softmax regression calculates the probability that a given input x belongs to each possible class. - For each class j , softmax regression computes a score z_j by taking the dot product of the input features with a corresponding weight vector w_j and adding a bias term b_j :
 - $z_j = w_j^T x + b_j$
- **Softmax Function****: - After computing the scores z_j for each class, the softmax function is applied to obtain normalized probabilities:
 - $$(z_j) = \frac{e^{z_j}}{\sum_{k=1}^K e^{z_k}}$$
 - - Here, K is the number of classes.

How Softmax regression works

- ****Prediction****: - The class with the highest probability output by the softmax function is chosen as the predicted class:
- $\hat{y} = \operatorname{argmax}_j(z_j)$
- **Compute Loss Function****: Calculate the categorical cross entropy loss: //we will discuss the entropy loss in the end of this lecture
- $$J(W, b) = -\frac{1}{N} \left[\sum_{i=1}^N \sum_{k=1}^K y_{ik} \log(\hat{y}_{ik}) \right]$$
- Where: - N is the number of training examples. - K is the number of classes. - y_{ik} is the indicator function, which equals 1 if example i belongs to class k and 0 otherwise. - $\hat{y}_{ik}$ is the predicted probability of example i belonging to class k . **It is not that**

How softmax regression works

- 4. ****Backpropagation****: Compute the gradients of the loss function with respect to the parameters W and b .

- Gradient for weights:

- $$\frac{\partial J}{\partial W} = \frac{1}{N} \sum_{i=1}^N (x^{(i)})^T (\hat{y}^{(i)} - y^{(i)})$$

- Gradient for biases:

- $$\frac{\partial J}{\partial b} = \frac{1}{N} \sum_{i=1}^N (\hat{y}^{(i)} - y^{(i)})$$

Note in Gradient Lecture we will show how to compute the gradient for MSE, Cross entropy , categorical cross entropy and the Softmax

How Softmax regression works

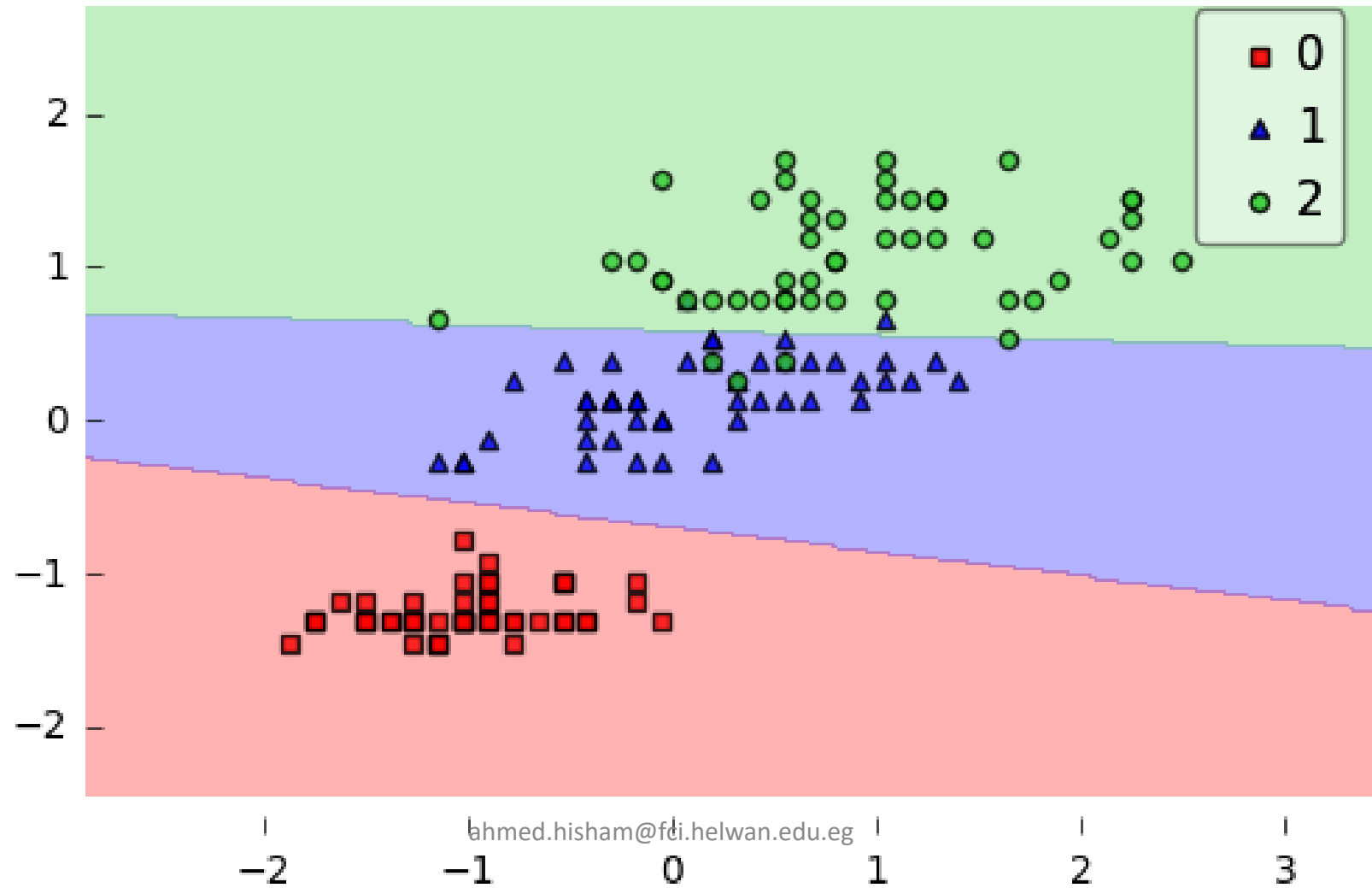
- 5. ****Update Parameters****: Update the parameters using gradient descent:
- $W := W - \alpha \frac{\partial J}{\partial W}$
- $b := b - \alpha \frac{\partial J}{\partial b}$
- Repeat these steps for multiple iterations until convergence. After training, you can use the trained model to make predictions on new data.

How Softmax regression works

- You can think of Softmax regression as multiple linear logistic regression where it work as binary classifier where (Class 1 , other classes) , (Class2 , other classes) and (class 3 , other classes)
- So if we want to classify 3 classes we will need three linear equation and our objective to find the best parameter for these three objectives function
- $z_0 = x_{11}w_{11} + x_{12}w_{12} + x_{13}w_{13} + \dots + b_1$
- $z_1 = x_{21}w_{21} + x_{22}w_{22} + x_{23}w_{23} + \dots + b_2$
- $z_2 = x_{31}w_{31} + x_{32}w_{32} + x_{33}w_{33} + \dots + b_3$

Softmax Regression Visualization

Softmax Regression - Gradient Descent



How Softmax regression works

Example

- a numerical example of softmax regression with three classes and three data samples. We'll calculate the scores, apply the softmax function, compute the loss, gradients, and update the parameters for one iteration.

Data Sample	Feature 1 (x1)	Feature 2 (x2)	Feature 3 (x3)	Class (y)
D1	1	2	3	0
D2	3	2	1	1
D3	2	3	1	2

How Softmax regression works

Example

- So our Hypothesis is to find the following three linear equation representing three different classes

- $z_0 = x_{01}w_{01} + x_{02}w_{02} + x_{03}w_{03} + b_0$

- $z_1 = x_{11}w_{11} + x_{12}w_{12} + x_{13}w_{13} + b_1$

- $z_2 = x_{21}w_{21} + x_{22}w_{22} + x_{23}w_{23} + b_2$

- $X = \begin{bmatrix} x_{01} & x_{02} & x_{03} \\ x_{11} & x_{12} & x_{13} \\ x_{21} & x_{22} & x_{23} \end{bmatrix}$

- $W = \begin{bmatrix} w_{01} & w_{02} & w_{03} \\ w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \end{bmatrix}$

- $b = \begin{bmatrix} b_0 \\ b_1 \\ b_2 \end{bmatrix}$

Important Note

Our objective to find the following three linear equation each equation represent a class to sperate this class from other classes

You can think of Softmax regression as multiple logistic regression where it work as binary classifier where (Class 1 , other classes) , (Class2 , other classes) and (class 3 , other classes)

How Softmax regression works

Example

1. ****Initialization****: - Initialize weights (W) and bias (b):

$$W = \begin{pmatrix} 0.1 & 0.2 & 0.3 \\ 0.4 & 0.5 & 0.6 \\ 0.7 & 0.8 & 0.9 \end{pmatrix}$$
$$b = \begin{pmatrix} 0.1 \\ 0.2 \\ 0.3 \end{pmatrix}$$

Note we calculated the z for first data samples (D1) only respect to each class
Repeat these process for other data samples

2. ****Calculate Scores****: - For data sample 1(D1):

$$z^1 = \begin{pmatrix} z_0|D1 \\ z_1|D1 \\ z_2|D1 \end{pmatrix} = \begin{pmatrix} 0.1 & 0.2 & 0.3 \\ 0.4 & 0.5 & 0.6 \\ 0.7 & 0.8 & 0.9 \end{pmatrix} \cdot \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix} + \begin{pmatrix} 0.1 \\ 0.2 \\ 0.3 \end{pmatrix}$$

$$z^1 = \begin{pmatrix} (0.1 \times 1 + 0.2 \times 2 + 0.3 \times 3) + 0.1 \\ (0.4 \times 1 + 0.5 \times 2 + 0.6 \times 3) + 0.2 \\ (0.7 \times 1 + 0.8 \times 2 + 0.9 \times 3) + 0.3 \end{pmatrix}$$
$$= \begin{pmatrix} 0.1 + 0.4 + 0.9 + 0.1 \\ 0.4 + 1.0 + 1.8 + 0.2 \\ 0.7 + 1.6 + 2.7 + 0.3 \end{pmatrix}$$

$$z^1 = \begin{pmatrix} z_0 \\ z_1 \\ z_2 \end{pmatrix} = \begin{pmatrix} 1.5 \\ 3.4 \\ 5.3 \end{pmatrix}$$

How Softmax regression works

Example

3. ****Apply Softmax Function****: - For each data sample, calculate probabilities using the softmax function: - For class 0:

$$P(y = 0|D_i) = \frac{e^{z_i}}{\sum_{j=0}^2 e^{z_j}}$$

- For class 1:

$$P(y = 1|D_i) = \frac{e^{z_i}}{\sum_{j=0}^2 e^{z_j}}$$

- For class 2:

$$P(y = 2|D_i) = \frac{e^{z_i}}{\sum_{j=0}^2 e^{z_j}}$$

Note the classes 0 ,1, 2 is mapped to one-hot encoded vector For example for first class will be [1 0 0] and second class [0 1 0] and for third class [0 0 1]

****Prediction****: - The class with the highest probability output by the softmax function is chosen as the predicted class:

$$\hat{y} = \operatorname{argmax}_j(z_j)$$

Compute Loss Function**: Calculate the categorical cross entropy loss:

$$J(W, b) = -\frac{1}{N} \left[\sum_{i=1}^N \sum_{k=1}^N y_{ik} \log(\hat{y}_{ik}) \right]$$

How Softmax regression works

Example

Data Sample 1**: - Scores:

$$z^1 = \begin{pmatrix} 1.5 \\ 3.4 \\ 5.3 \end{pmatrix}$$

- Probabilities:

$$P(y = 0|D_1) = \frac{e^{1.5}}{e^{1.5} + e^{3.4} + e^{5.3}} \approx 0.0191$$

$$P(y = 1|D_1) = \frac{e^{3.4}}{e^{1.5} + e^{3.4} + e^{5.3}} \approx 0.1275$$

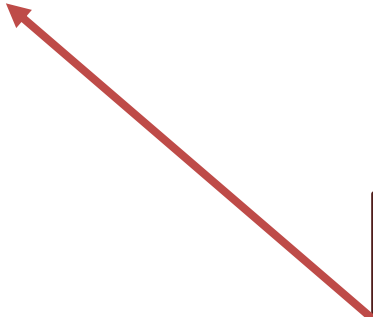
$$P(y = 2|D_1) = \frac{e^{5.3}}{e^{1.5} + e^{3.4} + e^{5.3}} \approx 0.8534$$

$$\text{So } \hat{y}^{(1)} = \begin{pmatrix} 0.0191 \\ 0.1275 \\ 0.8534 \end{pmatrix}$$

How Softmax regression works

Example

- - Actual Label: $y_1 = 0$ - Loss for data sample 1:
- $L_1 = - \sum 1 \times \log(0.0191) + 0 \times \log(0.1275) + 0 \times \log(0.8534)$
- $L_1 = -\log(0.0191) \approx 1.72$



Note the classes 0, 1, 2 is mapped to one-hot encoded vector. For example for first class will be [1 0 0] and second class [0 1 0] and for third class [0 0 1]

$$y^{(1)} = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}$$

How Softmax regression works

Example

- Repeat that with second data sample x_2 and calculate L_2
- Repeat that with the third data sample x_3 and calculate L_3
- The calculate total loss $(L_1+L_2+L_3)/3$
- $J(W, b) = -\frac{1}{N} \left[\sum_{i=1}^N \sum_{k=1}^N y_{ik} \log(\hat{y}_{ik}) \right]$

How Softmax regression works

Example

- Compute the gradients of the loss function with respect to the parameters W and b .

- Gradient for weights $\frac{\partial J}{\partial W}$:

- $\frac{\partial J}{\partial W} = \frac{1}{N} \sum_{i=1}^N (x^{(i)})^T (\hat{y}^{(i)} - y^{(i)})$

$$\frac{1}{3} \left[\begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix} \left[\begin{pmatrix} 0.0191 \\ 0.1275 \\ 0.8534 \end{pmatrix} - \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix} \right] + (x^{(2)})^T \left(\hat{y}^{(2)} - \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix} \right) + (x^{(3)})^T \left(\hat{y}^{(3)} - \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} \right) \right]$$

How Softmax regression works

Example

- And as same calculate the Gradient for biases $\frac{\partial J}{\partial b}$:
- $\frac{\partial J}{\partial b} = \frac{1}{N} \sum_{i=1}^N (\hat{y}^{(i)} - y^{(i)})$
- Update Parameters**: Update the parameters using gradient descent:
- $W := W - \alpha \frac{\partial J}{\partial W}$
- $b := b - \alpha \frac{\partial J}{\partial b}$

Derive Logistic regression from Softmax

- According to softmax For three classes we will need three linear equation
- $z_0 = x_{11}w_{11} + x_{12}w_{12} + x_{13}w_{13} + \dots + b_1$
- $z_1 = x_{21}w_{21} + x_{22}w_{22} + x_{23}w_{23} + \dots + b_2$
- $z_2 = x_{31}w_{31} + x_{32}w_{32} + x_{33}w_{33} + \dots + b_3$
- And for two classes we will need two equation
- $z_0 = x_{11}w_{11} + x_{12}w_{12} + x_{13}w_{13} + \dots + b_1$
- $z_1 = x_{21}w_{21} + x_{22}w_{22} + x_{23}w_{23} + \dots + b_2$
- But Actually we don't need the second equation as it considered binary classifier and one equation can separate between the two classes so we can assume that $z_1 = 0$

Derive Logistic regression from Softmax

- $P(y = 1|X) = \frac{e^{z_1}}{\sum_{j=0}^1 e^{z_j}} = \frac{e^{z_0}}{e^{z_0} + e^{z_1}} = \frac{e^{z_0}}{e^{z_0} + e^0} = \frac{e^{z_0}}{e^{z_0} + 1}$
- By dividing the numerator and denominator by e^{z_0}
- $P(y = 1|X) = \frac{1}{1 + e^{-z_0}} \rightarrow$ the logistic regression
- So we can consider the softmax regression is generalized form from Logistic Regression

References

- <https://ai.plainenglish.io/why-cant-we-use-linear-regression-to-solve-a-classification-problem-68edf1a3261b>
- <https://www.youtube.com/watch?v=YtebGVx-Fxw>
- https://www.graphpad.com/guides/prism/latest/curve-fitting/reg_simple_logistic_logodds.htm
- <https://medium.com/@matthewjacobholliday/an-introduction-to-log-odds-86d364fa6630>
- <https://www.geeksforgeeks.org/understanding-logistic-regression/>
- <https://www.geeksforgeeks.org/ml-linear-regression/>
- https://sphweb.bumc.bu.edu/otlt/mph-modules/bs/bs704-ep713_multivariablemethods/bs704-ep713_multivariablemethods5.html
- <https://www.analyticsvidhya.com/blog/2020/12/beginners-take-how-logistic-regression-is-related-to-linear-regression/>
- <https://medium.com/finswipe-en/why-dont-we-approach-to-classification-problems-using-linear-regression-in-machine-learning-8edcca89448>
- <https://www.linkedin.com/pulse/understanding-cross-entropy-loss-key-metric-problems-santhosh-sachin-dogac/>
- https://rasbt.github.io/mlxtend/user_guide/classifier/SoftmaxRegression/